ORACLE

User Manual

Online Readout & Acquisition for Collinear Laser spectroscopy Experiments

Version 0.6.0

Ethan Barnes
ethan.barnes@postgrad.manchester.ac.uk

November 10, 2026

Contents

1 Introduction **3**

2 Installation **3**

3 Quick start **4**
3.1 A first scan, on the simulated rack 4
3.2 A shift on the beamline 4

4 Scan controls **5**
4.1 Scan parameters 5
4.2 Buncher controls 5
4.3 Starting and stopping 6
4.4 Scan queue 6
4.5 Plots 6
4.6 ELOG, quick tune and the experiment information 6

5 Calibration and the gate **7**
5.1 Step and confirm 7
5.2 The gate 7
5.3 The Scan Calibration tab 7

6 Hardware Calibration **8**
6.1 Settle time 8
6.2 Linearity 8
6.3 Cooler monitor 9
6.4 Scan table 9
6.5 Health check 9

7 Scan Performance and Time graphs **11**
7.1 Scan Performance 11
7.2 Time graphs 11

8 Settings **12**

9 Data and records **13**
9.1 Run files 13
9.2 Files beside a run 14
9.3 Bench records and tables 14

A What ORACLE closes in the old DAQ **15**

B Troubleshooting and known limits **15**

References **16**

1 Introduction

ORACLE is the data acquisition for collinear laser spectroscopy (CLS) at IGISOL. It steps the platform voltage through a scan, records every ion the time tagger sees against the voltage it was taken at, and calibrates the voltage axis before every run. It keeps the collinear DAQ's layout, file format and functions, brings in the bench calibration tool cls-cal [1], and is built so that the faults found in the old DAQ cannot happen rather than being caught afterwards (Appendix A). Runs are read by cls_tools [2] and fitted in DENIS [3], both unchanged.

The window has five tabs above the quick-tune controls and the experiment information:

Scan controls. Scan and buncher parameters, run control, the scan queue, the ELOG panel and the live plots (Section 4).

Scan Calibration. A stand-alone calibration and the gate's verdict on it; every scan's own calibration is shown here too (Section 5).

Hardware Calibration. The bench tool's measurements of the rack itself: settle time, linearity, the cooler, the bench table and the health check (Section 6).

Scan Performance. How the rack's behaviour during a run shaped its spectrum, check by check, with the run's systematic inputs (Section 7).

Time graphs. Counts against time, to watch the beam.

Principles. Every calibration point is written, confirmed and only then read, so the voltage axis cannot slip. Nothing is cut: what deserves a look is flagged, and the operator decides in DENIS. Every input survives a crash, and a run can be reopened exactly as it was taken.

Status. ORACLE runs end to end on its simulated rack, which imitates the rack the bench tool measured in October 2026. It has not yet driven the beamline: its drivers follow code that has run on the rack but are tested against stand-ins for the vendor libraries.

Scope. This is a user manual. The code (`cls_acquisition/`, `gui/`) documents itself for developers; the README describes the test suite.

2 Installation

ORACLE runs on Windows and macOS (and Linux, simulated only). The installers use `uv`, which brings its own Python, so none need be installed.

```
git clone https://github.com/EthanBarnes54/ORACLE.git
cd ORACLE
install.bat full # Windows
./install.sh full # macOS / Linux
```

Option	Does
`full`	uv, the dependencies and a desktop shortcut (on macOS an `ORACLE.app` in `~/Applications`, linked from the Desktop)
`rack`	as `full`, plus the instrument libraries (pyvisa, pyepics, py-elog, spinapi, mcculw): for the beamline PC
`uv`	uv alone
`shortcut`	the shortcut alone

With no option the installer offers a menu. Launch from the shortcut, or with `uv run python gui.py`. To update, `git pull` then `uv sync`.

Simulated or hardware. ORACLE starts on the simulated rack: the status bar says **Simulated rack**, runs go to `data/simulated`, and nothing is posted to the ELOG. On the beamline PC, install with `rack` and set `rack: hardware` in `settings/settings.yaml` (Section 8).

The time tagger's wrapper. The collinear DAQ's compiled TimeTagger4 wrapper has four faults (Appendix A). Build the corrected one on the rack PC, with Visual Studio's C++ build tools and the cronologic driver installed:

```
cd cls_acquisition\hardware\timetagger4_wrapper
uv run --with pybind11 --with setuptools python build_wrapper.py
```

then set `instruments: tagger_driver` to that folder. Until it is built, the status bar carries a rack warning naming the inputs affected.

Where things are kept. `settings/` holds `settings.yaml` (yours), `inputs.yaml` (the window's last inputs) and `rack.yaml` (the last rack of each kind); `data/` the runs and records (Section 9); `logs/` a session log per launch, the newest 20 kept. File > Open data folder and Open log folder open them.

3 Quick start

3.1 A first scan, on the simulated rack

1. Launch ORACLE. On **Scan controls**, tick the first range and set **From (V)** 15 and **To (V)** 85, **Stepsize (V)** 1, **Bunches per channel** 25 and **Scan type** Sawtooth.
2. Press **Start scan**. ORACLE calibrates the scan's own setpoints, puts the calibration through the gate, writes the run's header and starts. The plots fill as hits arrive (Figure 1); the simulated beam has lines at 32, 47 and 71 V.
3. Press **Stop after scan** to end at the end of the scan in progress, or **Stop now!** to end within a bunch.
4. The run is analysed as it closes: **Scan Performance** gives its verdict, and **Scan Calibration** its calibration.
5. File > Open run... (Ctrl+O, Cmd+O on macOS) brings any run back as it was taken.

3.2 A shift on the beamline

1. Read the status bar. A rack warning (a fault ORACLE cannot mend, or hardware changed since the last session) is named there, its sentence in the tooltip.
2. Run the **Health check** (**Hardware Calibration**), with **Check the bench table first** ticked: the table, the cooler's ripple, each PMT's dark counts, the triggers and both meters' trigger sources, set against the last check.
3. If the hardware changed, measure the settle model (**Settle time**) and the bench table (**Scan table**) again before taking data.
4. On **Scan controls**, set the **Experiment name**, the **Interaction region** and the **PMT HV**, and the buncher timing.
5. Take scans one at a time, or lay them out in the **Scan queue** and let it run unattended.
6. Read each run's **Scan Performance** verdict, and anything flagged for review, before fitting it in DENIS.

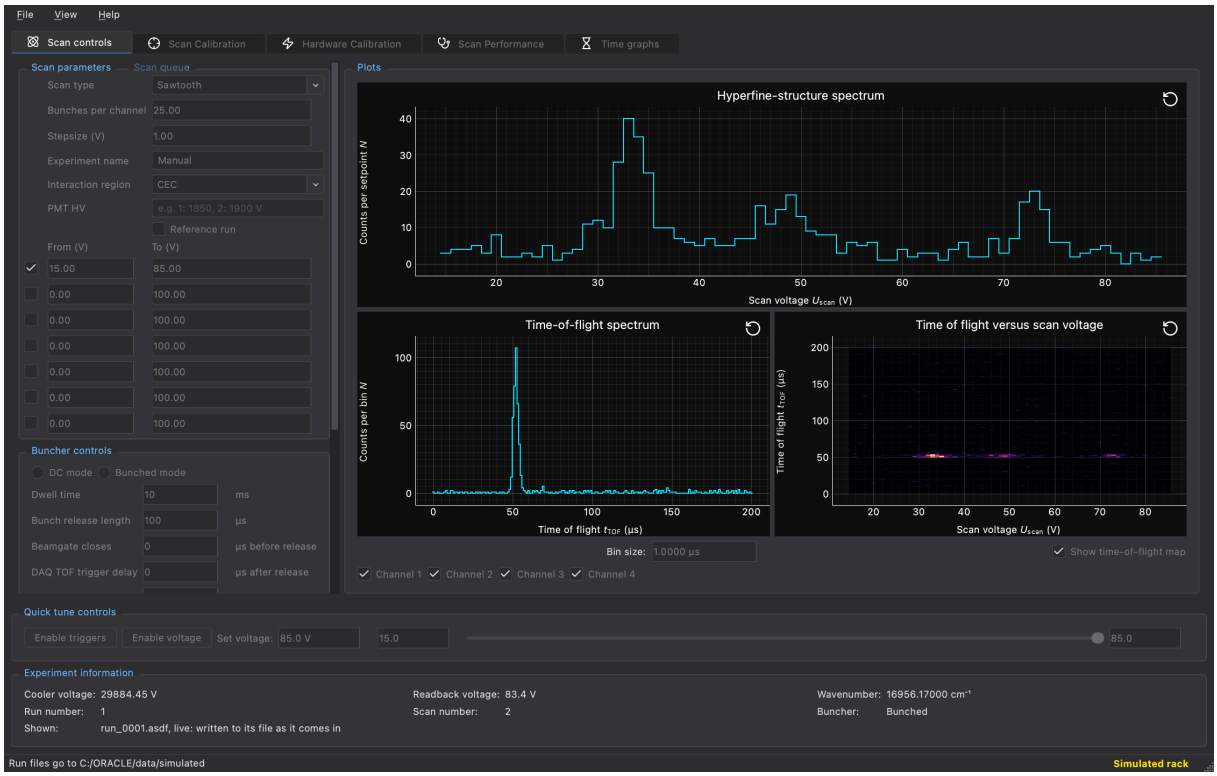


Figure 1: The **Scan controls** tab during a simulated scan: parameters on the left, the hyperfine-structure spectrum, time-of-flight spectrum and map on the right, quick tune and the experiment information below.

4 Scan controls

4.1 Scan parameters

Field	Meaning
Scan type	Sawtooth (repeats up), Triangle (up and back) or Single scan
Bunches per channel	bunches taken at each setpoint; **Seconds per channel** in DC mode
Stepsize (V)	the step between setpoints
From (V), **To (V)**	up to seven ranges, each ticked to be scanned; a point shared by two ranges is visited once
Experiment name	recorded with every run and in the ELOG
Interaction region	CEC or LCR: the cooler offset differs between them
PMT HV	the PMTs' high voltage, recorded where EPICS does not give it
Reference run	marks a run taken to measure the beam's offset from the cooler

Every voltage is held to the platform's limit (by default the Trek 609E-6's ±4 kV). A range that is not a whole number of steps ends on its last whole step, never past its stop.

4.2 Buncher controls

Bunched mode takes one bunch per **Dwell time**: the buncher is released for **Bunch release length**, and the tagger's window opens **DAQ TOF trigger delay** after the release and stays open from **DAQ gate starts** to **DAQ gate stops**, inside the tagger's 8.39 ms range. **DC mode** opens a window every millisecond; its gate stays settable, since DC hits come through the tagger too. A timing that does not fit its cycle (a release or delay past its end, a window still open at the next trigger) is refused with a sentence before anything starts.

4.3 Starting and stopping

Start scan calibrates the scan's own setpoints (Section 5); only if the gate passes are the run file written and the scan started. A refused calibration writes nothing, and its run number is given to the next attempt. **Stop after scan** finishes the scan in progress; **Stop now!** ends within a bunch.

At each step the tagger's window is shut, the DAC written, and the window held shut for as long as the settle model says the jump takes; every step then takes its full count of bunches at a settled voltage, while the buncher and cooler run on untouched. A tagger that sees no trigger for three cycles (two seconds at least) is reported on the status bar, and the scan holds its setpoint until triggers return.

4.4 Scan queue

The second page of the parameters box lays scans out ahead, one row and one run file each: ranges (`20:55, 60:80`), step, bunches (seconds in DC), order (Sawtooth or Triangle) and number of scans; timing, experiment and view are the parameters page's. Each row is checked as it is typed (a bad cell turns red) and the readout gives each row's setpoints and time, or its fault, and when the queue ends. **Add current** adds the parameters as they stand, **Show** puts a row back on them, rows paste from a spreadsheet, Delete clears cells.

Automatic starts each scan as the last ends, so a queue can run overnight; **Manual** asks before each scan after the first. Anything but a clean end (a calibration refused, a file not written, a scan stopped by hand) stops the queue with the platform at rest, its row saying why. **Stop queue after this scan** lets the running scan finish. Rows and states survive a crash; a restarted queue takes up at the first scan not done.

4.5 Plots

The hyperfine-structure spectrum, the time-of-flight (TOF) spectrum and, with **Show time-of-flight map**, the TOF against scan voltage. Ctrl-drag (Cmd-drag on macOS) on the TOF spectrum sets the gate, a double-click removes it; the spectrum and map count only hits inside it. **Bin size** and the **Channel** boxes choose the TOF binning and the PMTs. The view is held from Start to the scan's last event and recorded with the run.

Every plot handles alike: drag to pan, right-drag to stretch, the wheel to zoom (over an axis, that axis alone), Shift-drag to zoom to a box. A plot handled by hand stops following the data until its ↻ button, a double-click or View > Autoscale all plots (Ctrl+0) returns it. Resting the cursor on a title says what the plot shows and how it is calculated.

4.6 ELOG, quick tune and the experiment information

Send additional text appends to the open ELOG entry, **Start new entry** begins the next, **Send HFS screen** attaches the spectrum. Each scan posts its parameters and calibration; a simulated rack posts nothing.

Enable triggers runs the triggers without a scan; **Enable voltage** puts **Set voltage** on the platform, set by value or by the slider across its range, starting from the voltage already there. Spin boxes and the slider take the mouse wheel only once clicked, so scrolling past them never moves the platform. Below, the cooler and readback voltages, the wavenumber, run and scan numbers, the buncher mode, and **Shown**: the run every tab is showing, live, saved or opened.

5 Calibration and the gate

5.1 Step and confirm

Every run starts with a calibration of its own setpoints: each commanded platform voltage is paired with what the line meter reads through the ×1000 divider. For each setpoint, in order,

1. the acquisition process writes the DAC and confirms the write;
2. the meter waits for the point to settle, then takes *one* reading on command;
3. the reading is kept only if it was taken at least that long after its write; otherwise the reading (not the write) is asked for again after the shortfall.

Reading i cannot be asked for before write i is confirmed, so setpoint and reading cannot slip apart. Each reading integrates whole power line cycles (`calibration: nplc`, 1 by default), which takes out the mains a short reading catches. Retries are capped at three a point and ten a calibration; any other fault stops the calibration with a sentence naming it.

Settling. SETTLE, 50 ms, covers the amplifier's measured settling (a 2 kV jump within 0.5 V in 20.5 ms). Where the **Settle time** page has measured a settle model, each jump waits 1.25 times its settling, if longer.

5.2 The gate

A **REJECT** writes no file and starts no scan; a **WARN** goes to the ELOG with the run. The thresholds are `THRESHOLDS` in `cls_acquisition/calibration.py`, and each run records those it was judged by.

Check	Refuses (or warns) when
length	there is not one reading per planned setpoint
tracking	the readback does not follow the command ($\lvert r\rvert < 0.90$)
span	the readback's span is more than 10 % off the commanded span
polarity	(WARN) the two converters' lines meet 0 V more than 0.10 V apart
settle	a kept reading came sooner after its write than its jump takes to settle; (WARN) setpoints were read again
held	the last setpoint, read ten more times held 0.2 s on, scatters over 0.5 V, or the calibration's last reading sits more than half a step (or six of their scatter) from it: a lag of k puts it k steps off
plan	the commanded voltages are not the plan, once each, in order
dac_calibration	a measured DAC correction is past its validity; (WARN) one is in force, since the bench table is applied after collection

5.3 The Scan Calibration tab

A calibration over any range, without a scan: set **From (V)**, **To (V)** and **Stepsize (V)** and press **Start calibration**. The readback against setpoint, its deviation and the measured step fill point by point; the gate's verdict and every check follow (Figure 2). A scan's own calibration and an opened run's are shown here too.

Against the bench table. With a bench table in force (Section 6.4), every calibration is fitted to it, $V = a + bT$ at each point's DAC code, weighted by the meter's noise and the table's 10 mV repeatability. The report gives a (the run's 0 V level), $b - 1$, the residual against what those account for (WARN at twice that: the table is stale or the hardware changed), how many points fell between the table's codes, and a and b of the last runs on the same table. The fit is recorded with the run (`TableFit`); the header's `CalSet` and `CalReadback` are unchanged.



Figure 2: Scan Calibration: a run's calibration and the gate's report on it.

6 Hardware Calibration

The bench calibration tool cls-cal [1] in ORACLE: measurements of the rack itself, one page each, laid out as the other tabs (parameters, buttons and result on the left, plots on the right). Above the pages, **Ramp between points** sets the pace of every unmeasured move, and **Line meter: Read (5 × NPLC 10)** reads the platform precisely, to check the wiring and polarity against quick tune.

Every measurement drives the DAC by step and confirm, ramps back to 0 V however it ends, and writes its record under `<data>/hardware_calibration/` as it goes, so a stopped or failed one keeps what it took and is analysed as far as it went. On the rack each first asks the operator to arm the high voltage, naming the most it will apply. **Open record...** shows a record saved earlier, here or on another PC.

6.1 Settle time

Steps the platform out and back while the line meter samples on its own timer, and reports each step's lag, settling and 10–90 % rise, overshoot and noise; optionally a **Drift tail** at a long integration. **Use as scan settle model** puts the result in force for scans; **Use recommended wait for linearity** sets the sweep's wait to three times the slowest step (at least 0.2 s).

6.2 Linearity

A count-based ladder up, down, up then down, or in random order (which separates drift from shape), finer across 0 V, with 0 V read again every few rungs and several readings a point with autozero. It reports gain and offset, residual, curvature, each converter's line and where they meet 0 V, hysteresis, the 0 V drift and, from a dense sweep, each converter's integral non-linearity. **Use as DAC correction** puts a correction in force for **Valid for** days; production runs

write uncorrected, since the bench table is applied after collection, so a correction in force is warned of at every calibration.

6.3 Cooler monitor

Streams the cooler meter without gaps, at a short aperture (ripple) or an NPLC (drift), and reports level, drift, noise by decade, the white-noise floor, the oscillations (mains harmonics named, corrected for the meter's own attenuation) and the averaged spectrum.

6.4 Scan table

The bench table gives the platform voltage each DAC code really gives; each run's calibration is fitted to it (Section 5). On the bench tool's records, the 25 October table predicts the next day's sweep to 11.6 mV rms once $a + bT$ is fitted.

Measure table sweeps **Span** ±1 kV (about 1.5 h) or ±3 kV (about 5 h, overnight) in 1 V steps, 5 readings at NPLC 10 a point, 0 V every 10 points, **Order** up or random, at the Linearity page's settle time and watched there. The page warns while the 0 V checks say the rack is still warming up. The sweep is made into a table when it completes.

Resume… takes up a sweep that stopped, from its record, after its last 0 V check; the records make one table.

Check table (start of a shift, after a power cycle) reads 250 points across the table's span in a few minutes and fits them to it: PASS at 15 mV rms or less, else WARN, measure it again.

Make from a linearity record… builds a table from any sweep with 0 V checks. Tables are kept in `<data>/tables/`, named by date and ID (the hash of their contents), never overwritten.

Use a table…, Compare with…, Use none put one in force, set another against it code by code, or use none.

The page shows each converter's shape about a straight line, the 0 V level's drift and how often 0 V should be checked in a run (Figure 3). A table from another rack, or one the settings name that is missing, is warned of at start-up, never refused.

6.5 Health check

One report, at the start of a shift, on whether the rack is as it should be (Figure 4). With **Check the bench table first** and a table in force it runs the table check; then it reads the cooler for 10 s (its 50 Hz ripple), runs the Scan tab's triggers for 10 s counting each PMT (with the beam off, its dark counts) and the triggers, and reads both meters' trigger sources. Each part is set against the last check: WARN for ripple grown by half, a PMT silent or changed more than twice over, a trigger miscounted or lost, or a meter on its trigger input. Every check is kept as `hardware_calibration/health_<time>.yaml`; **Post to ELOG** posts it.

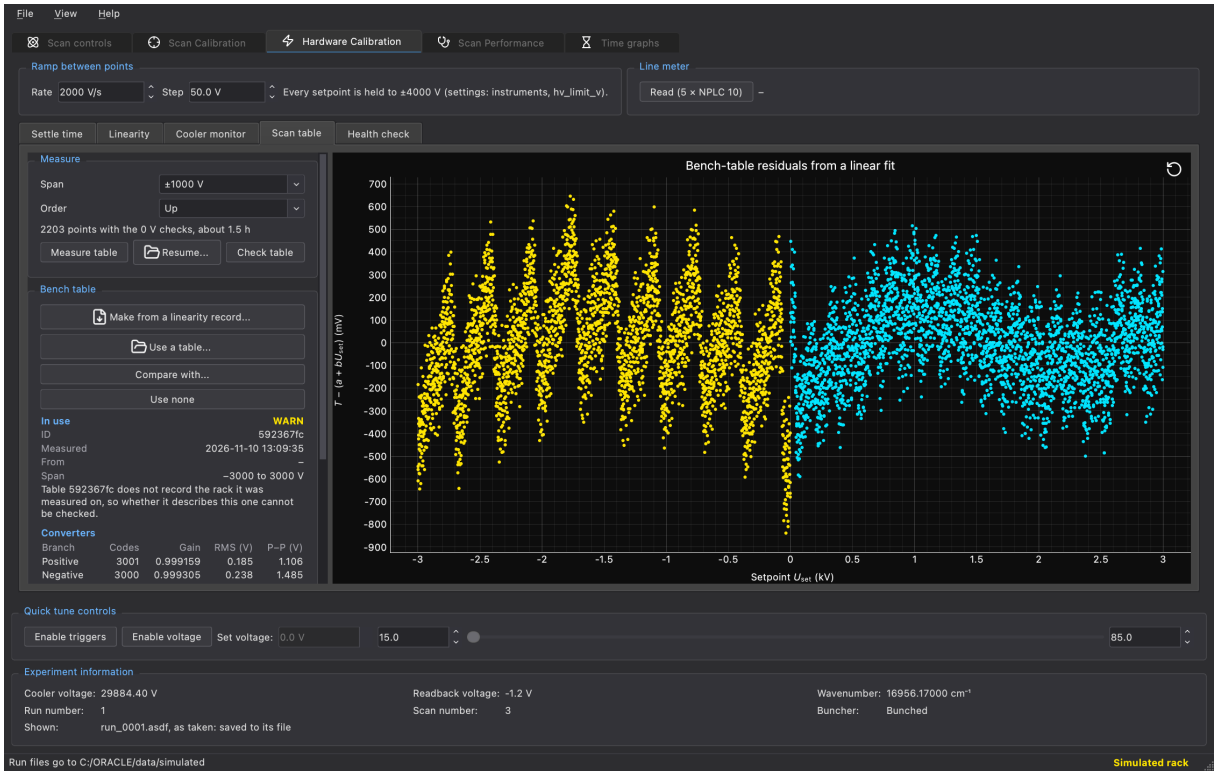


Figure 3: The **Scan table** page with the 25 October 2026 bench table in force: each converter's residual from a straight line.

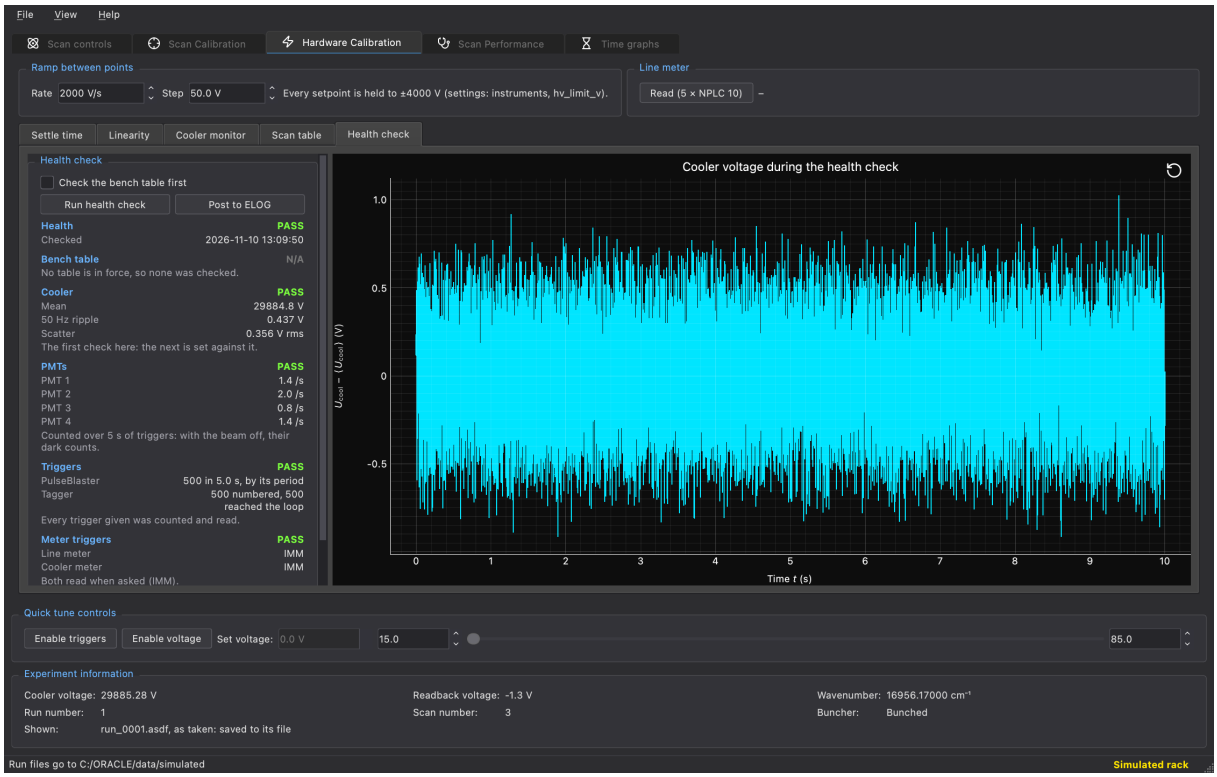


Figure 4: A health check on the simulated rack, and the cooler it read.

7 Scan Performance and Time graphs

7.1 Scan Performance

How the rack's behaviour during a run shaped its spectrum (Figure 5). With **Analyse each run as it ends** ticked, each run is analysed as it closes; any other is chosen under **Dataset** (the refresh button looks for new runs, the folder button takes one from anywhere) and shown on every tab by **Show and analyse**.

Every check gives PASS, WARN, FAIL or N/A with the number it decided on; the **Verdict** is the worst, with the line's width and what moved it. A deviation is flagged only when it is both improbable under counting statistics and large enough to matter against the line's width. Select a check to read what it found.

Voltage scan. Hits tagged with a voltage other than their bunch's; the calibration as DENIS fits it, and against earlier runs calibrated the same way; the HV during the scan against its calibration.

Timing. Whether the loop kept up with the triggers, stalls and pauses (a sleeping PC shows here), bunches taken before their step settled, missed triggers, the TOF peak against its gate.

Beam and spectrum. Beam losses and stability, the line's position scan by scan against what the HV and laser monitors predict, the spectrum's shape, each channel.

Sources. The cooler as the analysis takes it, a rolling mean over `cooler: rolling_s` (15 s), how far the readings scatter about it, and whether each hit's own reading is what its ions saw; the laser through the Doppler shift.

Detectors. Each PMT's afterpulsing: its counts within `dead_time_us` of its own beyond what the other PMTs show, its scatter across scans against dead time, and a dead time suggested for DENIS, never applied.

Flagged for review lists what deserves a look, with its reason; nothing is cut. **Systematic inputs** gives each source of error as a width at the line, in scan volts and MHz, marked measured, declared (in the settings) or not yet known; the Monte Carlo that turns them into errors is DENIS's. A short block goes to the ELOG as each run is analysed; **Post systematics to ELOG** posts the whole table. **Export report** writes `run_NNNN_performance.yaml` and `run_NNNN_systematics.yaml` beside the run.

7.2 Time graphs

Counts against time for the chosen channels, in bins of 100 s down to 0.1 s at the trigger rate in force, to watch the beam. Below, each PMT's afterpulse ratio as data arrive: its pairs within the dead time on itself over those on the others, 1 by chance, shown in the warning colour above 2.



Figure 5: Scan Performance for a simulated run: the verdict and checks, and a plot per systematic.

8 Settings

`settings/settings.yaml` holds what differs from the defaults in `cls_acquisition/settings.py`; it is written as settings are saved, and can be edited while ORACLE is closed. A file ORACLE cannot read (a Windows folder in double quotes, say) is refused at start-up with its line, never replaced by defaults.

Key	Meaning (default)
`rack`	`simulated` or `hardware`
`data_directory`	where runs go on the rack (`<app>/data`); a simulated rack writes to `<app>/data/simulated`
`experiment`	the experiment name, saved as it is edited
`elog`	`enabled`, `url`, `author` (disabled; never from a simulated rack)
`pvs`	EPICS names of the wavenumber, mass, mass setpoint and each input's PMT HV (`pmt_hv_1`–`4`)
`hardware`	what the rack cannot say: `scan_divider` and `cooler_divider` (`model`, `serial`, `ratio`, `ppm`; ratios 1000 and 10000) and the `amplifier`; recorded with every run
`instruments`	VISA resources of both meters, the MCC board (`dac_board`), the tagger driver's folder and thresholds, `hv_limit_v` (4000, the most allowed) and `settle_gate_bit` (the MCC line gating the tagger; empty: not wired)
`dac_correction`	a measured DAC correction (identity), written by **Use as DAC correction**
`scan_table`	the ID of the bench table in force (none)
`settle_model`	the scan's settling against jump size (none: SETTLE), written by **Use as scan settle model**
`calibration`	`nplc`: power line cycles a calibration reading integrates (1)
`cooler`	`rolling_s`: the window of the cooler's rolling mean (15 s)
`detectors`	`dead_time_us`: the dead time the PMTs are judged at (10 μs)
`systematics`	`beam_offset_v` by region as `[offset, error]`, `zero_drift_v` (0.030), `wavemeter_mhz` (10)
`laser`	harmonic (4), geometry (anti-collinear) and charge (1)
`cooler_pv`	publish the cooler as EPICS PVs (`FURIOS:Cooler:`; needs pcaspy)

Beside it, `inputs.yaml` keeps every input in the window, written half a second after a change and whole, so a crash leaves the last complete set; delete it to start from the defaults. `rack.yaml` keeps the last rack of each kind, so a change of hardware is named at start-up.

9 Data and records

9.1 Run files

Runs are written as the collinear DAQ wrote them, so cls_tools and DENIS read them unchanged: `<data>/<year>/<month>/<day>/run_NNNN.asdf`, ASDF standard 1.5.0 [4]. The header holds the archive's sixteen keys, checked as it is written (`CalSet` and `CalReadback` are the calibration), and the events follow as rows of

```
timestamp, voltage, bunch_number, channel, time, cooler
```

where `bunch_number` counts the scan's bunches from 0, exactly `BunchesPerChannel` a step, and `cooler` is the meter's reading in force for the hit (times 10000 for the platform). A run's number is one more than the highest in the data folder, so numbers survive a restart and a refused calibration leaves no gap.

One more header key, `ORACLE`, holds ORACLE's own record of the run; readers that look keys up by name pass over it, and File > Open run... restores the window from it.

Key	Contents
`Version`, `Commit`	ORACLE's version and git commit (`Hash`, `Dirty`)
`ScanType`, `Rounds`, `Queue`	the scan type, scans asked for, place in the queue
`Timing`, `View`	the trigger timing sent to the rack; the plots' view
`Calibration`, `CalibrationReadings`	the gate's verdicts and numbers; the readings' NPLC and the calibration's duration
`SettleModel`, `Laser`, `Channels`	the settle model, laser geometry and wired inputs
`CoolerRollingS`, `DeadTimeUs`	the cooler's rolling window; the PMTs' dead time
`Rack`, `RackIdentity`	simulated or hardware; the meters', DAC board's, tagger wrapper's and amplifier's identities
`Dividers`	both dividers as set; readers scale by these, never by today's settings
`DacCodes`, `DacCorrectionFull`	the converter and code each setpoint was written as; the DAC correction in full
`TaggerLevels`	each input's threshold
`ScanTable`, `TableFit`	the bench table in force; the calibration's fit to it
`InteractionRegion`, `ReferenceRun`, `PmtHv`	what the operator gave

9.2 Files beside a run

File	Contents
`run_NNNN_monitor.csv`	the rack every 0.1 s, written once a second: `time`, `round`, `setpoint_v`, `readback_v`, `cooler_v`, `wavenumber_cm1`, and the loop's `bunches`, `dropped`, `skipped`
`run_NNNN_performance.yaml`	the Scan Performance report: verdict, checks, flags, suggested dead times
`run_NNNN_systematics.yaml`	the systematic inputs

9.3 Bench records and tables

Hardware calibrations write CSV records under `<data>/hardware_calibration/`: `settle_*`, `linearity_*`, `cooler_*` and their companions, each with its settings as `#` lines above the columns, rows flushed as taken, and its status and results below; health checks as `health_<time>.yaml`.

Bench tables, proposed as the format for cls_tools, are kept as

`<data>/tables/table_<YYYYMMDD>_<id8>.csv`

one row per converter and code, `channel, code, setpoint_v, volts, sigma_v, readings`, where `channel` 0 drives the positive branch, `volts` is the platform voltage relative to the 0 V level, and `# key: value` lines (JSON values) above give its `id` (the SHA-256 of every number), when and on which rack it was measured, its source record, the dividers, the 0 V drift and the recommended 0 V check interval. A run's codes (`DacCodes`) and table (`ScanTable`) let its calibration be fitted as $V = a + b\,T(\text{code})$. It reads with

```
pandas.read_csv(path, comment='#')
```

A What ORACLE closes in the old DAQ

Fault in the collinear DAQ	In ORACLE
D1, Mode A: two threads on one meter could leave a burst ungated	One owner thread per instrument, and VISA's exclusive lock between programs; every reading is taken on command, never on the meter's trigger input (PulseBlaster channel 17), which each mode reads back
D2, Mode B: setpoint i read as reading $i + k$	Step and confirm: no lag by construction, and the held check refuses one from any route
D3, Mode C: a fixed shape read as a seam at 0 V	The platform runs on through 0 V; the shape is each converter's gain and non-linearity, measured and fitted by the bench table
D4: the DC path recorded nothing	DC hits come through the tagger
D5: a read that timed out moved the scan on	A timed-out read is no bunch; a silent tagger is reported and the scan holds
Mode B's root: one bunch per tagger read	Each trigger is a bunch by the card's own number; a skipped number is a lost trigger, reported
D6: plans gained a point	Plans are counted, then laid down; never past the stop
D7: nothing inspected a calibration	The gate, before anything is written
D8, D9: two sessions on one instrument; dead code	One session per resource; not carried over
A dead acquisition process hung the window; a crashed window left the rack held	One reply per instruction, a dead process reported; the process watches its parent and leaves the platform at 0 V
The tagger wrapper: inputs 3–4 on input 2's edge, rollover marks as hits, failed opens printed, empty triggers unmarked	The corrected wrapper (Section 2); a rack warning until it is built
Overload marks passed for voltages; meters left streaming by another program	Every reading checked; meters reset and identified on opening
The cooler's PV server opened the meter a second time	The PVs come from ORACLE's own readings
The bench tool lost an aborted sweep; its cooler blocks left gaps	Records written as they go; one stream

B Troubleshooting and known limits

Every refusal is a sentence naming the fault and its numbers; the session log (File > Open log folder) keeps it with everything else.

Seen	What to do
Rack warning on the status bar	Read its tooltip: a fault ORACLE cannot mend (the old tagger wrapper) or hardware changed since the last session (measure the settle model and the bench table again)
Run rejected after Start scan	The gate's failing check is named (Section 5); nothing was written. A *held* or *settle* refusal points at the meter or the settling; repeat, and measure the settle time
A timing refused before starting	Shorten the gate, release or delay, or lengthen the dwell, as the sentence says
No triggers reach the time tagger	Check the PulseBlaster, its cable to the tagger and the trigger level; the scan holds until triggers return
A meter shows NaN	It stopped answering its poll: check its USB and that no other program holds it
The settings are refused at start-up	Fix the named line (on Windows, use single quotes around a folder, or forward slashes)
The acquisition process stopped	The run is closed on what it recorded; restart ORACLE
A queue stopped	Its row says why; fix it and start the queue again, which takes up at the first scan not done

Known limits.

- Nothing has yet run on the beamline; treat each hardware path as unverified until bench-tested on the rack PC.
- SETTLE comes from bench steps up to 2 kV; the beamline's settle model should be measured across the jumps scans make, including the largest range join.
- The settle gate (an AND unit on PulseBlaster channel 18) is not wired: settling bunches are dropped in software, to within the USB latency.
- The DAC's codes are 0.305 V apart, so a 1 V step carries up to ±0.15 V of rounding that no correction removes; a step of a whole number of codes has none.
- The DAC correction drifts (offsets by up to 0.1 V a day): keep its validity short.
- The live view keeps every hit of the run in memory, 11 bytes a hit: about 40 MB an hour at 1000 hits a second.

References

[1] A. Raggio, *cls-cal*: bench calibration tool for the IGISOL CLS rack (settle time, linearity, cooler monitor), 2026.

[2] A. Raggio, *cls_tools*: readers and sorting utilities for IGISOL CLS run files, https://github.com/andry3vi/cls_tools.

[3] A. Kayaalp, *DENIS*: Doppler Estimation and Numerical Inference for Spectroscopy, https://github.com/ardakayaalp/DENIS.

[4] The ASDF standard, version 1.5.0, https://asdf-standard.readthedocs.io.